

# BERT4SessRec: Content-Based Video Relevance Prediction with Bidirectional Encoder Representations from Transformer

Xusong Chen

Dong Liu\*

University of Science and Technology  
of China  
Hefei, China

Chenyi Lei

Alibaba Group  
Hangzhou, China

Rui Li

Zheng-Jun Zha

Zhiwei Xiong

University of Science and Technology  
of China  
Hefei, China

## ABSTRACT

This paper describes our solution for the Content-Based Video Relevance Prediction (CBVRP) challenge, where the task is to predict user click-through behavior on new TV series or new movies according to the user's historical behavior. We consider the task as a session-based recommendation problem and we focus on the modeling of the session. Thus, we use the Bidirectional Encoder Representations from Transformer (BERT) methodology and propose a BERT for session-based recommendation (BERT4SessRec) method. Our method has two stages: in the pre-training stage, we use all sessions as training data and train the bidirectional session encoder with the masking trick; in the fine-tuning stage, we use the provided click-through data and train the click-through prediction network. Our method achieves session representations with the help of BERT, which effectively captures the bidirectional correlation in each session. In addition, the pre-training stage makes full use of all sessions, overcoming the positive-negative imbalance problem of the click-through data. We report the results of using different kinds of features on the test set of the challenge, which verify the effectiveness of our method.

## CCS CONCEPTS

• Information systems → Personalization.

## KEYWORDS

Bidirectional Encoder Representations from Transformer (BERT); deep learning; session-based recommendation

### ACM Reference Format:

Xusong Chen, Dong Liu, Chenyi Lei, Rui Li, Zheng-Jun Zha, and Zhiwei Xiong. 2019. BERT4SessRec: Content-Based Video Relevance Prediction with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 27th ACM International Conference on Multimedia (MM '19), October 21–25, 2019, Nice, France*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3343031.3356051>

\*Corresponding author: Dong Liu ([dongeliu@ustc.edu.cn](mailto:dongeliu@ustc.edu.cn)).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions.acm.org](https://permissions.acm.org).

MM '19, October 21–25, 2019, Nice, France

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6889-6/19/10...\$15.00

<https://doi.org/10.1145/3343031.3356051>

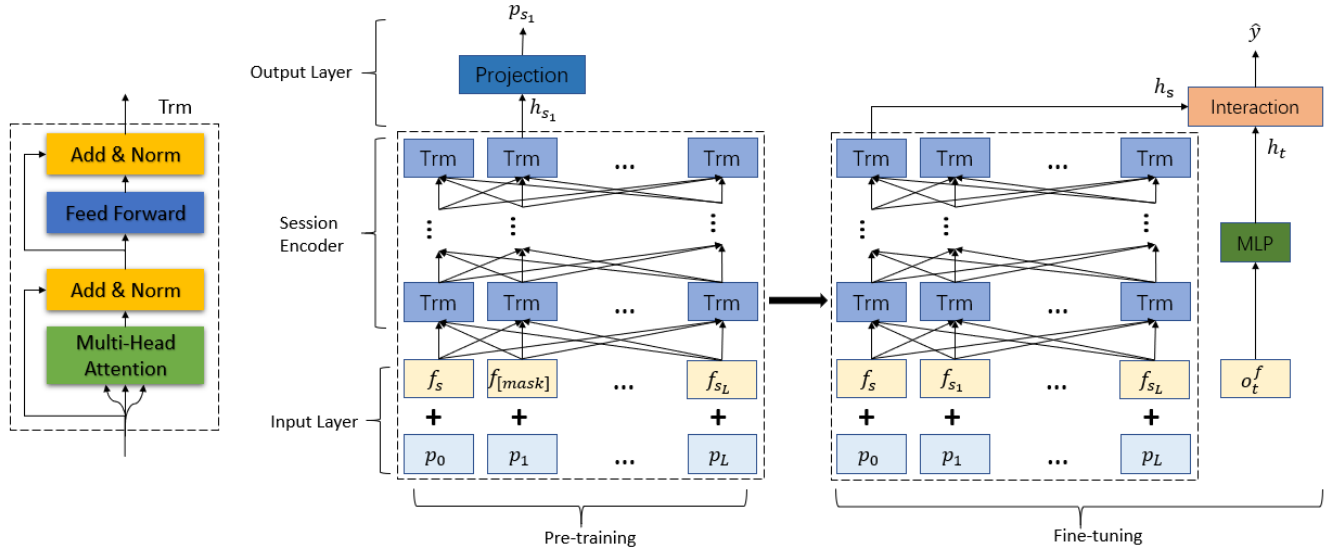
## 1 INTRODUCTION

We describe our solution for the Content-Based Video Relevance Prediction (CBVRP) challenge. The task of this challenge is to predict the user's next action (whether he/she will click a video or not) based on the user's historical behavior and the video content. For example, there is a sequence of video IDs: "486, 612, 2015, 402, 497, 1309, 560, 864, 445, 1795" and the target video ID is "1116", we aim to predict whether the user will click the target video or not according to the given sequence. Here, we do not know the exact user ID of the sequence, so we can regard the sequence as an anonymous session. In short, this task can be viewed as a session-based recommendation problem.

In recent years, Recurrent Neural Networks (RNNs) have led to great success in session-based recommendation [4, 6, 9] due to the power of RNNs in sequence modeling. For example, Hidasi *et al.* [4] applied recurrent neural networks with Gated Recurrent Units (GRU) into session-based recommendation. Tan *et al.* [9] further improved RNN-based model by employing two techniques: data augmentation and a method to account for shifts in the input data distribution. Li *et al.* [6] proposed Neural Attentive Recommendation Machine (NARM) by combining attention mechanism with GRU to improve the performance of session-based recommendation. The basic paradigm of these RNN-based methods is to represent anonymous session as a vector using a left-to-right pattern and make recommendations by computing the similarity between the representations of target video and anonymous session.

Based on our findings from analyzing the dataset, we find that the left-to-right pattern is not enough to represent the session. For example, in the anonymous session "486, 612, 2015, 402, 497, 1309, 560, 864, 445, 1795" in the training set of Track 1, we observe that adjacent item pair "560, 864" appears in the training set 13,801 times. The reverse item pair "864, 560" also appears in the training set 12,297 times. If we only consider left-to-right pattern, we can capture the relation from 560 to 864, but fail to capture the relation from 864 to 560 in this anonymous session. Thus, it is more reasonable to consider both left-to-right and right-to-left patterns simultaneously in the same session, which is to capture the bidirectional relation and co-occurrence.

Recently, there is a breakthrough in the field of natural language processing: Devlin *et al.* [2] proposed Bidirectional Encoder Representation from Transformer (BERT), a language representation model. BERT has significantly outperformed other state-of-the-art language models, e.g. OpenAI GPT [7], on eleven natural language



**Figure 1: Overview of the proposed method. Left: Details of the transformer layer (Trm for short); Middle: Pre-training with the masking trick; Right: Fine-tuning.**

processing tasks, such as question answering, sentence classification, and sentence tagging. Very recently, Sun *et al.* [8] adopted BERT for sequential recommendation, achieving the state-of-the-art performance.

Inspired by this, we apply BERT for session-based recommendation. Specifically, our method has two stages: (1) pre-training: we utilize BERT with the masked language model to train a session encoder, which allows us to use all sessions (including the sessions in the test set) without extra supervised information. The pre-training is unsupervised, as it randomly masks out some videos in sessions and the objective is to predict the masked videos based only on context; (2) fine-tuning: after pre-training of the session encoder, we use the click-through data (provided in the training and validation sets) to fine-tune our model in order to learn the relation between session representation and target video.

It is worth noting that BERT4Rec [8] is a generative method, which generates the representation of target video from user’s historical behavior, and then computes similarity between the generated representation of target video and all candidate videos to make recommendation. Our method is a discriminative method, where the representation of target video is learned from content features (e.g. audio, frame etc.) and the representation of session is learned by our pre-trained session encoder. And we train the interaction layer to calculate the similarity between the representations of target video and session. Furthermore, due to the imbalance of the click-through data (nearly 1:13 of positive and negative samples in the training set of track 1), it leads to poor generalization of the model and is easy to over-fit if we directly use the click-through data as supervised information to train. Our proposed method overcomes the data imbalance problem in the pre-training stage. Extensive experimental results and superior performance on both tracks of the challenge demonstrate the effectiveness of the proposed method.

## 2 OUR PROPOSED METHOD

In this section, we firstly give an overview of our method, then introduce deep bidirectional self-attention model and detail our pre-training and fine-tuning stages. Figure 1 provides an illustration of our method.

### 2.1 Overview

As illustrated in Figure 1, in pre-training stage, our model consists of input layer, session encoder and output layer. In the input layer, we prepare initial representation of video and then feed it into session encoder. Our session encoder is stacked by multiple self-attention layers, which are also referred to as Transformer layers [10]. In the output layer, we get the hidden representation of masked video and use it to predict the classification probability (details in Section 2.4). In fine-tuning stage, we directly employ pre-trained input layer and session encoder to initialize corresponding parts, instead of training from scratch. Finally, we make interaction between hidden representation of session and target video to predict whether user clicks target video or not. Note that we use all session data including sessions in test set in pre-training stage. In fine-tuning stage, we use the click-through data in training set and validation set as supervised information to fine-tune our model.

### 2.2 Input Layer

As illustrated in Figure 1, given an input session  $s$  of length  $L$ , we seek a manner to convert each video  $s_i$  in the session  $s$  at position  $i$  into a lower-dimensional space.

In the challenge data, each video corresponds to three kinds of content feature: audio, frame and video feature. Due to the various length of content feature for each video, we first adopt average pooling to form a feature vector  $o_{s_i} \in \mathbb{R}^{d_o}$  of video  $s_i$ , where  $d_o$  is the dimension of feature vector. Each video is corresponding to

a feature vector, thus we can form a feature matrix  $O \in \mathbb{R}^{N \times d_o}$ , where  $N$  is the total number of videos. We then perform principle component analysis and whitening operation on feature matrix  $O$  to get the new feature matrix  $O^f \in \mathbb{R}^{N \times d_o}$ . We keep all dimensions (no dimensionality reduction) and get the feature representation  $\sigma_{s_i}^f \in \mathbb{R}^{d_o}$  of video  $s_i$  from feature matrix  $O^f$ . Then, we project  $\sigma_{s_i}^f$  into a lower-dimensional space by learning a transformation matrix  $E_f$ , i.e.  $f_{s_i} = E_f \sigma_{s_i}^f$ , where  $f_{s_i} \in \mathbb{R}^d$  is a  $d$ -dimensional vector.

In addition, we also introduce embedding feature, i.e. we do not use any of the provided content feature, but we train a feature vector for each video from scratch.

Furthermore, according to BERT, each item is related to a positional embedding  $p_i \in \mathbb{R}^d$ , which indicates the position  $i$  within the session  $s$  to capture sequential order information, because Transformer layer is not aware of the order of the input session.

In order to get the final representation of session, we place a special feature vector  $f_s \in \mathbb{R}^d$  and its corresponding positional embedding  $p_0$  at the first position of each session. This  $f_s$  is invariant for different sessions. Since the Transformer layer will exchange information across all positions in the session, the final representation at this position can encompass the information of the entire session. It will be used as the session representation  $h_s$  in fine-tuning stage for click-through prediction.

At last, we employ element-wise addition operation to form initial representation of video  $s_i$ , i.e.  $h_{s_i}^0 = f_{s_i} + p_i$ . Accordingly,  $h_{s_i}^0 \in \mathbb{R}^d$ .

### 2.3 Session Encoder

As illustrated in Figure 1, session encoder consists of multiple Transformer layers, each Transformer layer consists of three parts: multi-head attention, feed forward layer and layer normalization.

**Multi-Head Attention.** Multi-head attention adopts scaled dot-product attention, which maps a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is the weighted sum of the values, where the weight corresponding to each value is computed by

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (1)$$

where  $Q \in \mathbb{R}^{n_Q \times d}$ ,  $K \in \mathbb{R}^{n_K \times d}$ , and  $V \in \mathbb{R}^{n_V \times d}$  denote queries, keys, and values respectively, where  $n_Q$ ,  $n_K$ , and  $n_V$  refer to the numbers of representations in the three inputs respectively, and  $n_K = n_V$ . Instead of computing the attention only once, multi-head mechanism applies the scaled dot-product attention  $h$  times in parallel. The independent attention outputs are simply concatenated and linearly transformed into the expected dimensions:

$$\begin{aligned} MH(H_s^m) &= [\text{head}_1; \dots; \text{head}_h]W^O \\ \text{head}_i &= \text{Attention}(H_s^m W_i^Q, H_s^m W_i^K, H_s^m W_i^V) \end{aligned} \quad (2)$$

where  $W_i^Q \in \mathbb{R}^{d_h \times d}$ ,  $W_i^K \in \mathbb{R}^{d_h \times d}$ ,  $W_i^V \in \mathbb{R}^{d_h \times d}$ , and  $W^O \in \mathbb{R}^{d \times d}$  are parameter matrices to be learned,  $h$  is the number of attention heads and set to 4,  $H_s^m \in \mathbb{R}^{(L+1) \times d}$  is the video hidden representation at layer  $m$  of the entire session  $s$ .

**Feed Forward Layer.** The multi-head attention sub-layer is mainly based on linear projections. To endow the model with nonlinearity and interactions between different dimensions, we apply feed forward layer to each video separately and identically. This consists of two linear transformations with a Gaussian error linear unit (GELU) activation in between.

$$\begin{aligned} FFN(x) &= \text{GELU}(xW_1 + b_1)W_2 + b_2 \\ \text{GELU}(x) &= x\phi(x) \end{aligned} \quad (3)$$

where  $\phi(\cdot)$  is the cumulative distribution function of the Gaussian distribution,  $W_1 \in \mathbb{R}^{d \times 4d}$ ,  $W_2 \in \mathbb{R}^{4d \times d}$ ,  $b_1 \in \mathbb{R}^{4d}$  and  $b_2 \in \mathbb{R}^d$  are parameters and shared across all videos. We omit the layer subscript  $m$  for convenience, but in fact, these parameters are different from layer to layer.

**Layer Normalization.** As described above, it is natural to consider capturing more complex item bidirectional relation by stacking multiple Transformer layers. However, it is difficult to train model as it goes deeper. For better training, we adopt a residual connection [3] around two sub-layers in Figure 1, followed by layer normalization [1]. We can iteratively stack  $M$  Transformer layers to revise the representation of each video and the hidden video representation of layer  $m$  can be written as

$$\begin{aligned} H_s^m &= \text{Trm}(H_s^{m-1}), \forall i \in [1, \dots, M] \\ \text{Trm}(H_s^{m-1}) &= \text{LN}(A_s^{m-1} + \text{FFN}(A_s^{m-1})) \\ A_s^{m-1} &= \text{LN}(H_s^{m-1} + \text{MH}(H_s^{m-1})) \end{aligned} \quad (4)$$

### 2.4 Pre-training

As elaborated above, we get the final hidden representation  $H_s^M$  at layer  $M$  for all videos in current session  $s$ . In pre-training stage, we adopt BERT with the masked language model to train our input layer and session encoder. Specifically, we randomly mask out  $\rho$  percentage of videos in the session  $s$ , and then predict those masked videos based on the remaining context. For example, in Figure 1, we mask the first video of the session. In general, we replace the video  $s_i$  at position  $i$  in session  $s$  with a special token [mask], where [mask] is corresponding to a special vector  $f_{[\text{mask}]} \in \mathbb{R}^d$  (it is also invariant in different sessions). Thus, the feature  $f_{s_i}$  of video  $s_i$  is replaced with  $f_{[\text{mask}]}$ , which is illuminated in Figure 1. Finally the hidden representation  $h_{s_i}^M$  of position  $i$  is fed into the projection layer followed by softmax,

$$p_{s_i} = \text{softmax}(\text{GELU}(h_{s_i}^M W_3 + b_3)O^f + b_o) \quad (5)$$

where  $W_3 \in \mathbb{R}^{d \times d_o}$  is parameter matrices to be learned,  $b_3$  and  $b_o$  are biases.  $O^f$  is the feature matrix that has been defined before (in Section 2.2). In case of embedding feature,  $O^f$  consists of the embedding features of all videos.  $p_{s_i}$  is the predicted probability of masked video  $s_i$  in session  $s$ . Finally, the negative log likelihood loss is employed to train our model.

$$L_P = - \sum_{s_i \in S} \log(p_{s_i}) \quad (6)$$

where  $S$  the set of masked videos.

**Table 1: Results on the test set (courtesy of challenge organizers). Late fusion indicates the simple average of the outputs of different features.**

| Method       | Track 1: TV series |               |               |               |               | Track 2: Movies |               |               |               |               |
|--------------|--------------------|---------------|---------------|---------------|---------------|-----------------|---------------|---------------|---------------|---------------|
|              | Audio              | Frame         | Video         | Embedding     | Late fusion   | Audio           | Frame         | Video         | Embedding     | Late fusion   |
| CBF          | 0.5104             | 0.5126        | <b>0.5222</b> | /             | 0.5166        | 0.5325          | 0.5408        | 0.5326        | /             | 0.5460        |
| NARM         | 0.5695             | 0.4910        | 0.4871        | /             | 0.5177        | 0.5732          | 0.5787        | 0.5250        | /             | 0.5888        |
| BERT4SessRec | <b>0.6305</b>      | <b>0.6002</b> | 0.4871        | <b>0.6618</b> | <b>0.6485</b> | <b>0.5968</b>   | <b>0.6497</b> | <b>0.6339</b> | <b>0.6608</b> | <b>0.6594</b> |

**Table 2: Results on the test set (courtesy of challenge organizers).**

| Method         | Track 1: TV series | Track 2: Movies |
|----------------|--------------------|-----------------|
| Our submission | 0.664490           | 0.652329        |

## 2.5 Fine-tuning

For our task, we take the final hidden state at position 0 at layer  $M$  as the final representation of the current session. We denote this vector by  $h_s \in \mathbb{R}^d$ . As illuminated in Figure 1, we feed the target video feature  $o_t^f$  selected from the feature matrix  $O^f$  into a Multilayer Perceptron (MLP) to get new representation  $h_t \in \mathbb{R}^d$ . When we use embedding feature for BERT, we omit the MLP and use the embedding feature directly as  $h_t$ . The session representation  $h_s$  and target video representation  $h_t$  are fed into the interaction layer to predict final click-through probability  $\hat{y}$ , where interaction layer can be inner product or more complex such as concatenation operation followed by MLP. Finally, the objective function used in fine-tuning stage is the sigmoid cross-entropy loss function.

## 3 EXPERIMENTAL RESULTS

We use the dataset provided by the challenge organizers, including Track 1 and Track 2. More details about the dataset can be found in the webpage<sup>1</sup>. We test different methods on the test set, send our predictions to the challenge organizers and get the evaluation results (in terms of AUC). Besides our proposed method, we compare with the following two methods.

- CBF: This is a simple baseline. The content features of videos in the session are averaged, then concatenated with the target video feature, and fed into a MLP to predict the click-through probability.
- NARM [6]: An RNN-based state-of-the-art model which employs attention mechanism to capture main purpose from the hidden states and combines it with the sequential behavior as final representation to generate recommendations.

All the methods including ours and the compared baselines are implemented with TensorFlow and running on an NVIDIA GTX 1080Ti graphical processing unit. The following hyper-parameters are used for all the methods, if not specially noted. The batch size is set to 128. Adam optimizer [5] is adopted for training. Learning rate is 0.001 in pre-training stage. In fine-tuning stage, we set learning rate to 0.001 and 0.0001 for track 1 and track 2 respectively. Once the validation loss does not decrease in two consecutive epochs, we divide the learning rate by 2. We also early stop by observing

the validation performance. The maximal number of epochs is 50. But in fine-tuning stage, we often stop in less than five epochs. To prevent over-fitting,  $L_2$  regularization coefficient of all models is tuned in the range of  $[0.0, 10^{-7}, 10^{-6}, \dots, 1]$  for the best performance. The mask percentage  $p$  is set to 0.1 and 0.2 in track 1 and track 2 respectively (note that the session length is 10 and 5 in the two tracks respectively). The feature dimension  $d$  in our model is set to 64. The number  $M$  of Transformer layers is set to 2. Interaction layer adopts concatenation operation followed by MLP ( $128 \rightarrow 64 \rightarrow 1$ ).

Table 1 presents the experimental results for measuring performance of different methods. Firstly, we observe that our proposed method outperforms CBF and NARM by a large margin, except using video feature in track 1. It verifies the effectiveness of our proposed method. Secondly, it can be observed that only using embedding feature leads to the best performance in our proposed method. The reason might be that embedding encodes more latent semantic information than content feature. In practice, if encountering new video that has no embedding feature (cold-start), we can still rely on the content feature.

## 3.1 Challenge Submission

We try a lot of combinations of different kinds of feature, different models, different training strategies, etc., and perform model ensemble. For track 1, our submitted results are achieved by late fusion of an improved NARM with audio feature and a BERT model with embedding feature. For track 2, we adopt the late fusion of a NARM model with audio and frame feature and a BERT model with embedding feature. For both tracks, we pre-train a lot of models and choose several according to the validation performance, and then fine-tune the chosen ones. We employ either inner product or concatenation operation followed by MLP as interaction layer, and use different dropout ratios  $\{0.0, 0.1, 0.2\}$  in fine-tuning stage. At last, we average the results of different models. The performance of our submission is shown in Table 2. It is worth noting that some results shown in Table 1 are achieved after the challenge submission.

## 4 CONCLUSION

We have presented a deep bidirectional self-attention model BERT for the content-based video relevance prediction challenge. Experimental results show the effectiveness of the proposed method. In the future, we plan to study the BERT model with mixed content feature and embedding feature.

## ACKNOWLEDGMENTS

This work was supported by the Natural Science Foundation of China under Grant 61772483.

<sup>1</sup><https://github.com/cbvrp-acmmm-2019/cbvrp-acmmm-2019>

## REFERENCES

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Vol. 1. 4171–4186.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
- [4] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
- [5] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [6] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM Conference on Information and Knowledge Management*. ACM, 1419–1428.
- [7] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. <https://www.cs.ubc.ca/~amuham01/LING530/papers/radford2018improving.pdf>.
- [8] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. *arXiv preprint arXiv:1904.06690* (2019).
- [9] Yong Kiam Tan, Xinxing Xu, and Yong Liu. 2016. Improved recurrent neural networks for session-based recommendations. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. ACM, 17–22.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. 5998–6008.